

Reviewer Pack — Subword Complexity of Lifted Rows Lower-Bounds Real Rank for...

Entry da476af6b27a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 19:01:00 UTC

Subword Complexity of Lifted Rows Lower-Bounds Real Rank for IND_2

Entry ID: da476af6b27a **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 19:01:00 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics on words / symbolic dynamics: subword complexity $p_w(n) = \#\{\text{distinct contiguous length-}n \text{ factors of } w\}$ and the Morse-Hedlund classification (Morse-Hedlund 1938; Cassaigne; Allouche-Shallit, 'Automatic Sequences' Ch.4). The Rauzy-graph factor-counting invariant is computable in $O(|w| \cdot n)$ by a single sliding-window pass. Distinct from Lempel-Ziv / RLE entropy (blacklisted) and from descent or inversion statistics; rarely deployed in lifting / communication complexity. **Field B** (complexity object): Deterministic two-party communication complexity of the indexing-lifted Boolean function $f \circ \text{IND}_2$, accessed via the real-log-rank lower bound $D^{\text{cc}}(f \circ \text{IND}_2) \geq \lceil \log_2 \text{rk}_R(M) \rceil$ on the canonical $2^k \times 4^k$ lifted communication matrix $M[x,y] = f(y_1[x_1], \dots, y_k[x_k])$ (Raz-McKenzie 1999; Göös-Pitassi-Watson 2015 lifting target at smallest nontrivial gadget arity $b=2$).

Statement:

For every Boolean function $f: \{0,1\}^k \rightarrow \{0,1\}$ with $k \leq 4$, build the lifted matrix $M = M_{\{f \circ \text{IND}_2\}} \in \{0,1\}^{2^k \times 4^k}$ and read each row r_x in lex order of $y \in (\{0,1\}^2)^k$ as a binary string of length 4^k ; let $s(f) := \max_{x \in \{0,1\}^k} p_{\{r_x\}}(k)$, where $p_w(n)$ is the subword complexity. Then $\text{rk}_R(M) \geq s(f)$, so $D^{\text{cc}}(f \circ \text{IND}_2) \geq \lceil \log_2 s(f) \rceil$. A single (f,x) with $p_{\{r_x\}}(k) > \text{rk}_R(M)$ refutes the conjecture.

Rationale (proposer's reasoning):

The IND_2 gadget injects a locally periodic alignment scaffold into every row of the lifted matrix, so two length- k windows of r_x that show distinct bit-patterns must witness distinct gadget-alignment cosets and we conjecture each contributes an independent direction to the column span. This pivots a classical Morse-Hedlund symbolic-dynamics invariant onto the lifted matrix in a way orthogonal to fooling-set, log-rank, and Möbius/Walsh attacks already tried. If true, subword complexity becomes a poly-time-computable, stable surrogate for partial log-rank that scales with gadget arity.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 91d595f7f1f9c7b8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 300 sampled instances (100 uniform random f for each $k \in \{2, 3, 4\}$), every instance must satisfy $\text{rk}_R(M) \geq s(f)$ with the rank computed by `numpy.linalg.matrix_rank` at tolerance $1e-8$ and $s(f)$ the max row subword complexity at window length k . SUP-PORTED iff zero violations; FALSIFIED on first (f, x) with $p_{\{r_x\}}(k) > \text{rk}_R(M)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - subword complexity communication complexity lifting indexing gadget - Rauzy graph factor complexity real rank Boolean matrix lower bound - Morse-Hedlund subword complexity log-rank lifted matrix Raz-McKenzie

Top relevant hits considered: - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise) - [<http://arxiv.org/abs/nlin/0101006v1>] On Complexity and Emergence - [<http://arxiv.org/abs/2401.1462>] Structure in Communication Complexity and Constant-Cost Complexity Classes - [<http://arxiv.org/abs/2508.119>] A robust family of residually finite groups; spectra of residual finiteness growth, computability properties, and other - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/1210.1461v1>] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/0903.1627v5>] On the Morse-Hedlund complexity gap

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def f(x):
        return [random.randint(0, 1) for _ in range(k)]
```

```

def IND_2(y):
    return [(y >> i) & 3 for i in range(k)]

def subword_complexity(s: str, n: int) -> int:
    seen = set()
    for i in range(len(s) - n + 1):
        window = s[i:i+n]
        if window not in seen:
            seen.add(window)
    return len(seen)

k_values = [2, 3, 4]
total_metric_value = 0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for k in k_values:
    for _ in range(100):
        f = [f(x) for x in range(2**k)]
        M = [[f[i][j] if (i & (1 << j)) else 0 for j in range(k)] for i in range(2**k)]

        rank_M = matrix_rank(M, tolerance=1e-8)
        rows = [''.join(str(bit) for bit in row) for row in M]
        p_r_x_values = [subword_complexity(row, k) for row in rows]
        s_f = max(p_r_x_values)

        if rank_M < s_f:
            conjecture_holds = False
            counterexample = f"First failing (f,x): {f}, {rows[0]}, {k}"
            break

        total_metric_value += rank_M / s_f
        instances_tested += 1

    return {
        "metric_name": "tightness_ratio",
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else 0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def matrix_rank(matrix, tolerance=1e-8):
    m = len(matrix)
    n = len(matrix[0])
    rank = 0

    for i in range(n):
        if all(abs(matrix[j][i]) < tolerance for j in range(m)):
            continue

        pivot_row = next(j for j in range(i, m) if abs(matrix[j][i]) > tolerance)

        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]

```

```

    for j in range(m):
        if i == j:
            continue
        factor = matrix[j][i] / matrix[i][i]
        for k in range(n):
            matrix[j][k] -= factor * matrix[i][k]

    rank += 1

return rank

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    total_metric_value = sum(result["metric_value"] for result in seeds if "metric_value" in result)
    instances_tested = sum(result["instances_tested"] for result in seeds if "instances_tested" in result)
    support_fraction = sum(1 for result in seeds if result["conjecture_holds"]) / len(seeds)

    if all("conjecture_holds" in result and result["conjecture_holds"] for result in seeds):
        print(f"RESULT: SUPPORTED mean={total_metric_value / instances_tested} std=0 support_fraction={support_fraction}")
    elif any("counterexample" in result for result in seeds):
        counterexample = next(result["counterexample"] for result in seeds if "counterexample" in result)
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if "counterexample" in result)
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7cc2688d.py", line 94, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7cc2688d.py", line 42, in run_trial
    f = [f(x) for x in range(2**k)]
        ^^^
TypeError: 'list' object is not callable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` ('list' object is not callable') due to a variable shadowing bug before any instances were evaluated, so no data was produced to assess the pre-registered support or falsification conditions. | next: Fix the name collision (rename the inner list, e.g., `truth_table = [f(x) for x in range(2**k)]`) and rerun the 300-instance sweep to obtain `violation_count` and mean tightness ratios.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	147130
2	preregistration	claude_max	opus	0	0	6108
3	novelty	claude_max	opus	0	0	3373
4	novelty	claude_max	opus	0	0	9650
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9912
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10550
7	judge	claude_max	opus	0	0	4510

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 191233 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/da476af6b27a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/da476af6b27a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/da476af6b27a.tar.gz` (if generated)